PROJECT PROGRESS REPORT

Hostel Buddy

Smart Hostel Complaint Management System

30% Implementation Milestone

Group 19

Department of Computer Science and Engineering

IIIT Kottayam

Milestone completed: 21 July 2026

1. Team Members

#	Member Name	Roll Number
1	Pranjal Tyagi	2024BCD0053
2	Dinesh Saini	2024BCD0033
3	Ajinkya	2024BCS0173
4	Saurabh	2024BCS0185
5	Vijay	2024BCS0249

2. Milestone Summary

This report documents the completion of the **30% implementation milestone** of the Hostel Buddy project. At this stage the project foundation is in place and the first functional module — user authentication (login) — is working end to end and verified by automated tests.

30%

Deliverable	Status	Completed on
Git repository	Completed	21 July 2026
Project skeleton	Completed	21 July 2026 (11:52)
Database setup	Completed	21 July 2026 (11:52)
Login module	Completed	21 July 2026 (12:00)

> **What "30%" means here.** The four deliverables above correspond to the first two increments of our phase-wise plan — **Phase 0 (Project Foundation)** and **Phase 1 (Authentication & Accounts)**. Both are committed to the project's Git repository and the login flow is covered by a passing integration test suite. The remaining 70% (complaint management, admin management, dashboards, full frontend, hardening) is scheduled across Phases 2–6.

3. Git Repository

A Git repository was initialised and connected to a remote on GitHub. All work is committed under the group's authorship, and the milestone corresponds to the first two phase commits.

Item	Detail
Repository	github.com/Pranjaltyagi76/Hostel-Buddy-College-Semester-5-Project
Version control	Git, hosted on GitHub (public)
Branch	`main`
Foundation commit	`8eef41d` — "Phase 0: project foundation" (21 Jul 2026, 11:52)
Login-module commit	`e4b145e` — "Phase 1: authentication & account management" (21 Jul 2026, 12:00)

Ignored from VCS	`node_modules/` , `.env` (secrets), `data/` (database file), `uploads/`

A `.gitignore` and `.gitattributes` were added so that dependencies, secrets and runtime data are never committed, and line endings stay consistent across the team's machines.

4. Project Skeleton

The project follows a **layered architecture**: each request passes through routes → middleware → controllers → services → repositories → database. The skeleton created at this milestone establishes that structure and a working server.

Component	Purpose
`src/server.js`	Application entry point — prepares the database, then starts the server
`src/app.js`	Wires middleware and mounts module routes
`src/config/`	Environment configuration (`env.js`) and fixed constants — categories, statuses, roles (`constants.js`)
`src/middleware/`	Central error handler and request logger (auth guard added with the login module)
`src/db/`	Database connection, schema and bootstrap
`package.json`	Node project definition, dependencies and run scripts (`npm start`)

Technology base: Node.js with the Express web framework. A health-check endpoint (`GET /api/health`) confirms the server boots correctly. The application was verified to start and respond successfully.

5. Database Setup

The database was set up using **SQLite** through Node.js's built-in `node:sqlite` module, which requires no separate database server and no native compilation — so every team member can run the project without extra installation. The schema is applied automatically on startup.

Two tables were defined. At the 30% milestone the **users** table is fully in use by the login module; the **complaints** table is created and ready for Phase 2.

Table	Key columns	Notes
users	id (PK), name, email (unique), password_hash, room_number, role, created_at	In active use — stores every account; `role` distinguishes student from admin; passwords stored only as bcrypt hashes
complaints	id (PK), user_id (FK→users), category, description, image_url, status, admin_remarks, created_at, updated_at, resolved_at	Created and ready; used from Phase 2 onward

The schema enforces integrity at the database level: a `UNIQUE` constraint on email prevents duplicate accounts, a foreign key ties each complaint to its author, and `CHECK` constraints restrict role, category and status to their defined values. Indexes were added to support later search and reporting.

6. Login Module (Authentication)

The first functional module — **user authentication** — is complete and working end to end. It covers student registration, login for both students and the administrator, and the protection of secured endpoints.

Capability	Description
Student registration	Creates an account with name, email, room number and password; rejects duplicate emails
Login (student & admin)	Verifies credentials and issues a signed session token (JWT)
Password security	Passwords are hashed with bcrypt and never stored or returned in plain text
Access control	Middleware verifies the token and the user's role before allowing access to protected routes
Admin account	A single administrator account is seeded automatically on first startup
Profile endpoints	View and update own profile; admin can list registered students

Implementation files: `modules/auth` (routes, controller, service), `modules/users` (routes, controller, service, repository), `middleware/auth.js` (route guards), `utils/jwt.js` and `utils/validators.js`, and `db/seedAdmin.js`.

Verification

The login module is covered by an automated integration test suite (`tests/auth.test.js`) that exercises the full flow. All checks pass, including the security-sensitive cases:

- Registration succeeds and returns a session token; duplicate email is rejected
- The password hash is never exposed in any response
- A wrong password returns a generic error that does not reveal whether the email exists
- Requests without a valid token are rejected; students cannot reach admin-only endpoints

7. Remaining Work (70%)

The remaining implementation is planned across five further increments:

Phase	Status	Scope
Phase 2	Planned	Complaint core — raise, view, edit/delete (Pending only), image upload
Phase 3	Planned	Admin complaint management — view all, search/filter, status workflow, remarks
Phase 4	Planned	Dashboards & analytics — student and admin statistics, charts
Phase 5	Planned	Frontend user interface — all pages and screens
Phase 6	Planned	Hardening & documentation — security, testing, final polish

Summary. At the 30% milestone (21 July 2026) the Git repository, project skeleton, database and login module are complete, committed and tested. The foundation and authentication layer that every later feature depends on is in place, keeping the remaining phases low-risk.